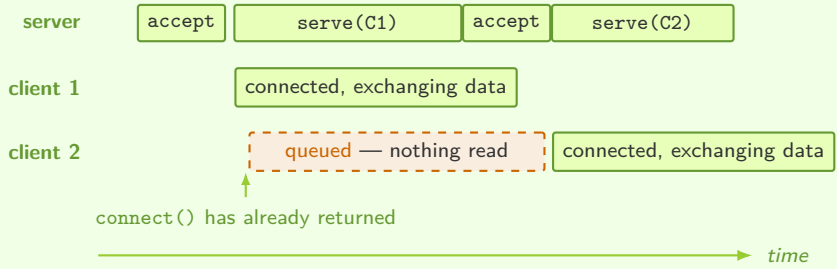
Processes: fork, Descriptor Inheritance and Signals

Lecture 09

Computer Networks and Distributed Programming

Where we are: one client at a time

Your chat server is correct and unusable: while it is inside `serve`, nobody else is being read.



The way out is to have something else — a second process — run `serve` while the server goes back to `accept`.

A process, a pid, a parent

Definition

A process is a running program: its own memory, its own descriptor table, its own place in the scheduler's queues.

- Every process has a pid, a small non-negative integer, and a ppid, the pid of the one that created it.
- `getpid()` and `getppid()` return them and cannot fail; both have type `pid_t`.
- Parenthood makes the processes on a machine a tree, whose root is the first process the kernel starts, pid 1.
- A pid is unique while the process lives, and is reused afterwards: it names a process, not a program.

fork: one call, two returns

One process calls it, two processes return from it.

```c
#include <unistd.h>

pid_t fork(void);

pid_t pid = fork();
if (pid < 0)  { perror("fork"); exit(EXIT_FAILURE); }
if (pid == 0) {
    /* child: fork returned 0 here */
} else {
    /* parent: fork returned the child's pid here */
}
```

The child is not told its own pid by `fork`: it asks for it with `getpid`. The parent is, because it will need it to wait for it.

What the child inherits

The child starts life as a duplicate of the parent, at the same point in the same code.

- Memory: a private copy of everything — globals, locals, heap. The kernel copies it lazily, but neither side can see that.
- Descriptors: every open descriptor is duplicated, and the two copies refer to the same open file. This is the important one.
- Signals: the child keeps the parent's handlers and its mask, until it installs its own or replaces the program.
- Not inherited: the pid, the ppid (parent pid), and any child the parent was waiting for: those are the parent's, not the family's.

Two copies of the same variables

```c
int glob = 1;                       /* one copy now, two after the fork */

int main(void) {
    int local = 10;
    pid_t pid = fork();
    if (pid == 0) { glob++; local++; }   /* the child, and only the child */
    else            sleep(1);            /* let the child print first */
    printf("pid=%d glob=%d local=%d\n", getpid(), glob, local);
    return 0;
}
```

```
pid=7183 glob=2 local=11      child
pid=7182 glob=1 local=10      parent
```

Nothing says which one runs first: here the `sleep` decides, and a `sleep` is not a synchronisation mechanism.

The line that gets printed twice

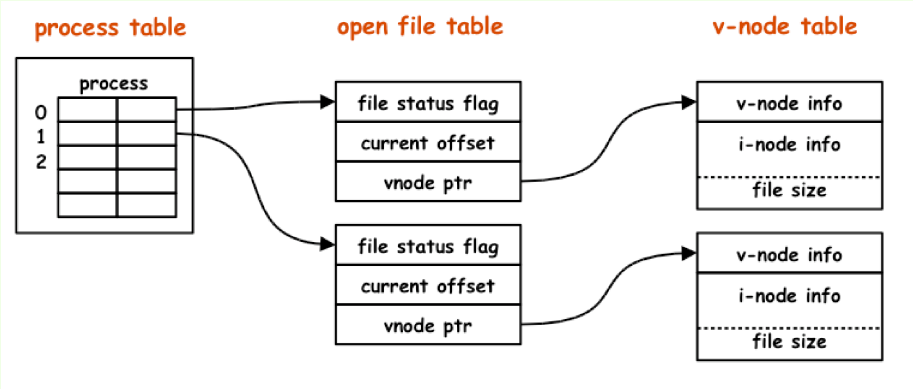
The program prints one line before forking. How many times does it appear?

On a terminal, once. Redirected to a file, twice — and not a character of the program has changed.

```
$ ./fork1 > out.txt ; cat out.txt
before the fork
pid=7189 glob=2 local=11
before the fork
pid=7188 glob=1 local=10
```

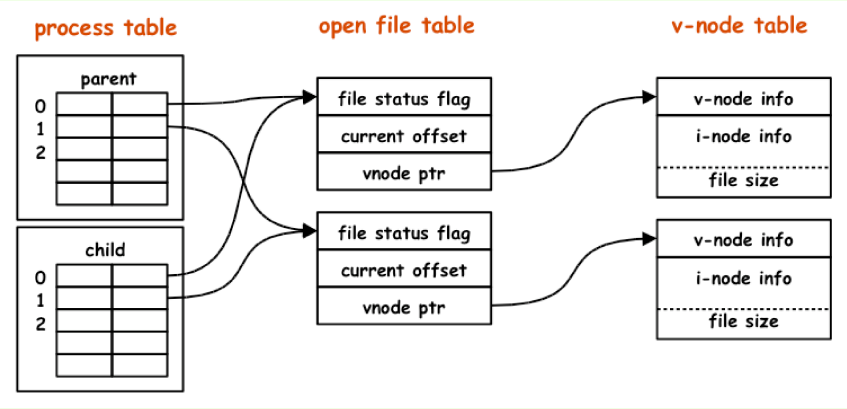
To a terminal `stdout` is flushed line by line; to a file it is not, so the line is still in the buffer when `fork` copies the memory — buffer included. Both processes flush it later.

Before the fork



One table per process, one entry per open — flags and offset live there, not in the process — and one object per file. Three levels, and the fork touches only the first.

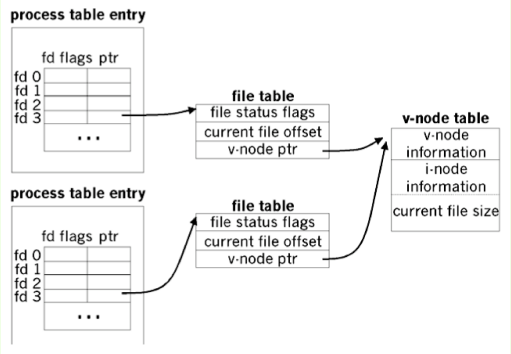
After the fork: one open file, two processes



The table is copied, the entries are not: `fd 3` in the parent and `fd 3` in the child are two arrows to one open file, with one offset and one reference count, now equal to two.

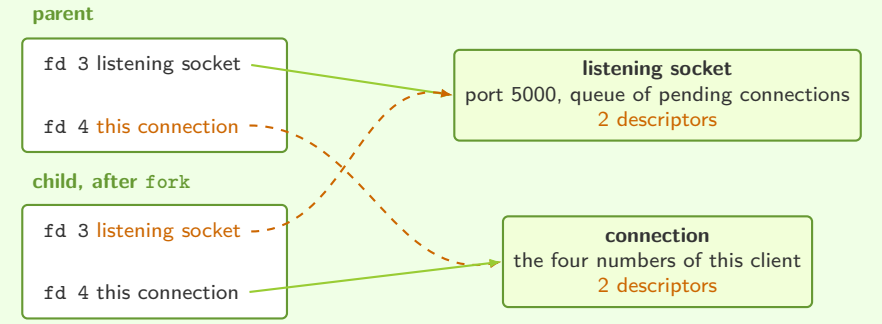
Two opens are not one fork

- Two unrelated processes that open the same file get one entry each: two offsets, and each one overwrites the other.
- Sharing does not come from the object being the same: it comes from the descriptor being inherited.
- `dup` makes a second descriptor onto the same entry inside one process — the same duplication, without a second process.



The connected socket is inherited too

A socket is one of those objects, so nothing here is new — except what it costs.



Dashed: the descriptor that process has to close. Each object stays alive until its last one goes, so the child cannot end the connection on its own, and the parent cannot either.

Who closes what

```c
for (;;) {
    int fd = accept(listen_fd, NULL, NULL);
    if (fd < 0) { perror("accept"); continue; }

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); close(fd); continue; }

    if (pid == 0) {                  /* the child serves this one client */
        close(listen_fd);           /* it will never accept anything */
        serve(fd);
        close(fd);
        _exit(0);                    /* not exit(): the buffers are not ours */
    }
    close(fd);                       /* the parent never talks on this connection */
}
```

Two closes that look redundant and are not: each side gives up the descriptor it does not use.

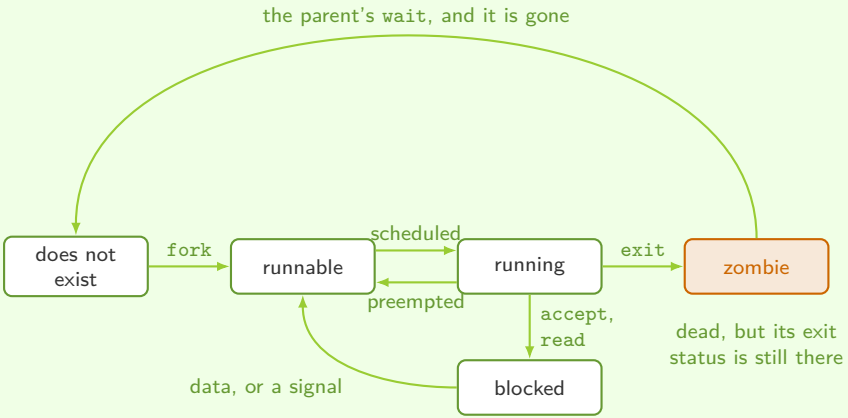
The parent that forgets to close

The child has finished and exited. Why is the client still waiting?

Because the connection is not closed: the parent still holds a descriptor on it, so the count never reaches zero and no `FIN` is sent.

- The client's `read` does not return 0: it blocks on a connection that nobody will ever write to again.
- Meanwhile the parent keeps one descriptor per client served, and `accept` eventually fails with `EMFILE`.
- The mirror image is just as bad: a child that keeps `listen_fd` holds the port open after the server is gone.

The life of a process



A server spends almost all its time blocked, and the interesting part is the last step: a dead child does not disappear until somebody asks how it died.

Three ways to end, and the status

- return from main: exactly the same as calling `exit` with that value.
- exit(n): runs the handlers registered with `atexit`, flushes the standard streams, then calls `_exit`.
- _exit(n): straight to the kernel: nothing is flushed. That is why a forked child uses it — the buffers it would flush are a copy of the parent's.
- Abnormally: an uncaught signal, or `abort`, which sends itself one.

Whatever the way, the kernel closes the descriptors and frees the memory, and keeps a termination status for the parent.

wait and waitpid

```c
#include <sys/wait.h>

pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);

int status;
wait(&status);                      /* any child; blocks until one ends */
waitpid(pid, &status, 0);           /* that child, and no other */
waitpid(-1, &status, WNOHANG);    /* any child, and never block */
```

- Both return the pid of the child they collected, or -1 with `ECHILD` if there is no child left to wait for.
- With `WNOHANG` the return is 0 when children exist but none of them has ended: nothing to collect, keep going.

Reading the termination status

Asking	Then
`WIFEXITED(status)`	`WEXITSTATUS` gives the value passed to `exit`
`WIFSIGNALED(status)`	`WTERMSIG` gives the signal that killed it
`WIFSTOPPED(status)`	`WSTOPSIG` gives the signal that stopped it

- `status` is not a number you print: it is a word the kernel packs, and only these macros read it.
- A child that called `exit(7)` leaves a `status` whose value is not 7 — only `WEXITSTATUS` says 7.
- Only the low eight bits of the value survive the trip, so exit codes live between 0 and 255.

Zombies

A process that has ended but has not been waited for is kept by the kernel: its memory is gone, its status is not.

```
$ ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
 4821  3310 S+   ./server
 4822  4821 Z+   [server] <defunct>
```

- It costs a slot in the process table, and a server that forks per client runs out of slots in an afternoon.
- You cannot kill it: it is already dead. It goes away when the parent waits, or when the parent does.
- An orphan is the opposite case — the parent died first, pid 1 adopted it, and pid 1 always waits.

A server cannot sit in wait()

Put `wait(NULL)` after the `fork`, in the accept loop. What have you built?

The sequential server again, with one extra process per client: the parent goes back to `accept` only when the child has finished serving.

- The parent must reach `accept` immediately, and still collect children that ended at some unpredictable moment.
- Polling with `WNOHANG` on every loop works, but only wakes up when a new client happens to connect.
- What is missing is a way for the kernel to say now, in the middle of whatever the process is doing.

A signal is an asynchronous notification

Definition

A signal interrupts a process to tell it that something happened. It carries a number and nothing else — no data, no sender, no queue.

- From the terminal: `Ctrl-C` sends `SIGINT` to every process in the foreground group, children included.
- From the kernel: an invalid memory reference, a division by zero, a child that ended, a write to a closed connection.
- From another process: `kill`, which sends a signal and is badly named: most signals do not kill anything.

It arrives between two instructions of your program, and you do not get to choose which two.

The signals a server meets

Signal	Sent when	Default
`SIGCHLD`	a child ended or stopped	ignored
`SIGPIPE`	you wrote to a connection the peer closed	terminate
`SIGINT`	`Ctrl-C` on the terminal	terminate
`SIGTERM`	`kill` with no signal named	terminate
`SIGQUIT`	`Ctrl-\` on the terminal	terminate, core
`SIGKILL`	somebody wants it gone now	terminate

Two of these are the whole reason a forking server needs signals at all: `SIGCHLD`, which is ignored by default and leaves you the zombies, and `SIGPIPE`, which kills the server the first time a client hangs up mid-reply. We come back to that one with robustness.

Ignore, catch, or let it kill you

For each signal a process has one disposition, and it is inherited across the fork.

- Default: whatever the signal says — for most of them, terminate the process, sometimes with a core file.
- Ignore: `SIG_IGN`: the signal is delivered and thrown away. For `SIGCHLD` this means something special, in a moment.
- Catch: a function of yours runs, and then the program resumes where it was interrupted.
- Neither: `SIGKILL` and `SIGSTOP` cannot be caught or ignored: they exist so that a process can always be stopped.

Installing a handler

```c
#include <signal.h>

static void on_child(int sig) { /* ... */ }

struct sigaction sa;
sa.sa_handler = on_child;         /* or SIG_IGN, or SIG_DFL */
sigemptyset(&sa.sa_mask);         /* nothing extra blocked while it runs */
sa.sa_flags   = 0;
if (sigaction(SIGCHLD, &sa, NULL) < 0) { perror("sigaction"); }
```

- The handler takes the signal number, so one function can serve several signals; install it once per signal.
- Old code says `signal(SIGCHLD, on_child)`. It is shorter and its exact behaviour differs between systems: use `sigaction`. Only to ignore a signal is `signal(sig, SIG_IGN)` still fine: you will meet it for `SIGPIPE`.

What a handler is allowed to do

It runs inside your program, between two of its instructions, and possibly inside a library call that is halfway through its own work.

- So it may only call async-signal-safe functions: `write` yes, `printf` and `malloc` no, however often you see them there.
- It must save and restore `errno`: the interrupted code was probably about to read it.
- The only variable it should share with the program is a `volatile sig_atomic_t`, read in the main loop.
- Keep it short. Its job is to record that something happened, not to handle it.

Reaping children with SIGCHLD

```c
static void reap(int sig) {
    int saved = errno;                          /* the handler must give it back */
    while (waitpid(-1, NULL, WNOHANG) > 0)
        ;                                       /* every child that has ended */
    errno = saved;
}
```

- Installed once before the accept loop, it collects children at the moment they die, and the parent never blocks.
- Simpler alternative: set `SIGCHLD` to `SIG_IGN` and the system creates no zombies at all — at the price of never learning how any child ended.
- While the handler runs, the `accept` the parent was blocked in returns `-1` with `EINTR`. Handling that belongs with making the server robust.

Three children die at the same moment

Three clients disconnect together. How many times does the handler run?

Once, maybe twice — never three times you can count on. Standard signals are not queued: pending is a flag, not a number.

- While the handler runs, further `SIGCHLD` are blocked; several of them collapse into the one still pending afterwards.
- A handler that calls `waitpid` once therefore leaves zombies behind, and the leak is slow enough to survive testing.
- Hence the loop: collect until `waitpid` says there is nothing more, not until you have collected one.

Sending signals: kill

A signal is sent by number, from the shell or from a program, to one process or to a whole group.

```
$ kill -TERM 4821
$ kill 4821       same thing
$ kill -INT 4821
$ kill -l         names, numbers
```

```c
int kill(pid_t pid, int sig);

kill(pid, SIGTERM);  /* one process */
kill(0,   SIGTERM);  /* my whole group */
kill(pid, 0);        /* nothing sent: */
                     /* does it exist? */
```

You may signal your own processes only. And since pids are reused, `kill(pid, 0)` answers a process with that pid exists, which is not quite the question you asked.

Stopping the server on purpose

```c
static volatile sig_atomic_t running = 1;       /* the only shared variable */

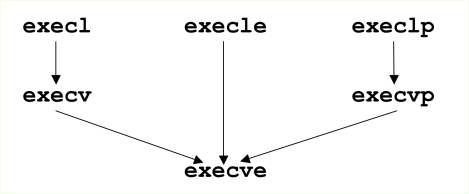
static void stop(int sig) { running = 0; }      /* records, does not act */

while (running) {
    int fd = accept(listen_fd, NULL, NULL);
    if (fd < 0) {
        if (errno == EINTR) continue;           /* a signal arrived: re-test */
        perror("accept"); break;
    }
    /* ... fork, serve, close ... */
}
close(listen_fd);                               /* say goodbye properly */
```

Without this the server dies inside `accept` and leaves its children running with the connections still open.

exec: the process becomes another program

- `exec` creates nothing: it replaces the program running inside the process that calls it.
- Text, data, heap and stack are thrown away and rebuilt from the executable; execution starts at its `main`.
- The pid, the ppid and the open descriptors stay: it is the same process, wearing another program.
- On success it does not return. Code after the call runs only when the call failed.



Five library functions, one system call

Six names for one call

Function	Program	Arguments	Environment
`execl`	path	list	inherited
`execlp`	name, via PATH	list	inherited
`execle`	path	list	given
`execv`	path	array	inherited
`execvp`	name, via PATH	array	inherited
`execve`	path	array	given

The letters: `p` search `PATH`, `l` arguments as a list, `v` arguments as a vector, `e` environment passed explicitly.

```
1 char *argv[] = { "gzip", "-c", "log", NULL };
2 execvp("gzip", argv);              /* the same as: */
3 execlp("gzip", "gzip", "-c", "log", (char *) NULL);
```

fork, exec, wait

```c
pid_t pid = fork();
if (pid < 0) { perror("fork"); return -1; }

if (pid == 0) {                       /* the child, and only for an instant */
    execlp("gzip", "gzip", "-c", path, (char *) NULL);
    perror("execlp");                 /* reached only if exec failed */
    _exit(127);
}

int status;
if (waitpid(pid, &status, 0) < 0) perror("waitpid");
if (WIFEXITED(status) && WEXITSTATUS(status) == 0) { /* it worked */ }
```

The program name appears twice: once to find the file, once because it becomes `argv[0]` of the new program.

Descriptors survive exec

Your server runs an external program. What does that program get?

Every descriptor the server had open, the listening socket included — and it can keep the port busy long after the server is gone.

- That inheritance is what makes redirection possible, so it is a feature; it is only a bug when you forget it.
- Mark the ones that must not travel with close-on-exec: they are then closed by `exec`, and only by it.

```
1 fcntl(listen_fd, F_SETFD, FD_CLOEXEC);   /* or SOCK_CLOEXEC at creation */
```

Summary

- `fork` returns twice: the child gets 0 and a private copy of the memory, and neither side knows who runs first.
- Descriptors are duplicated, not copied: two processes point at one open file, one connection, one reference count.
- So each side closes what it does not use, and a connection ends only when its last descriptor is closed.
- A dead child stays a zombie until somebody waits for it, and the server cannot afford to sit in `wait`.
- `SIGCHLD` is how the kernel says now; signals are not queued, so the handler collects in a loop.
- `exec` keeps the process and replaces the program — descriptors included, unless you say otherwise.

Try it

Two short programs

One: fork a child that exits immediately while the parent sleeps. Find both with `ps`, then add the `wait` and watch the zombie go. Two: a program that never ends by itself and prints a line every time it receives `SIGINT` or `SIGUSR1`.

```
$ ps -o pid,ppid,stat,cmd       the zombie is the one in state Z
$ ./waiter &
$ kill -USR1 <pid>              and Ctrl-C, and finally -KILL
```

Then answer: why does `Ctrl-C` reach the program even though you never told the terminal its pid?